Ideation Phase
Define the Problem Statements

Date	27.03.2026
Team ID	69bf8031c1eab70902258949
Project Name	Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets
Maximum Marks	

Problem Statement:

The Challenge:

In many ServiceNow environments, users frequently submit forms (such as Incident, Change, or Custom Application forms) without providing essential information. This occurs because the fields are not set as **Mandatory** by default, or they only become necessary under specific conditions (e.g., a "Resolution Note" should only be mandatory when the state changes to "Resolved").

Furthermore, making these changes directly in a **Production** environment is high-risk. There is a need for a standardized process to configure these dynamic UI requirements in a **Development** instance and move them safely through the landscape without manual re-configuration.

The Objective

- **Dynamic Enforcement:** Implement **UI Policies** to conditionally enforce mandatory fields based on user input or record state, improving data integrity without cluttering the form for all users.
- **Controlled Deployment:** Utilize **Update Sets** to capture these configuration changes, allowing for seamless migration, testing, and auditing across different ServiceNow instances (Dev $\rightarrow$ Test $\rightarrow$ Prod).

Technical Scope

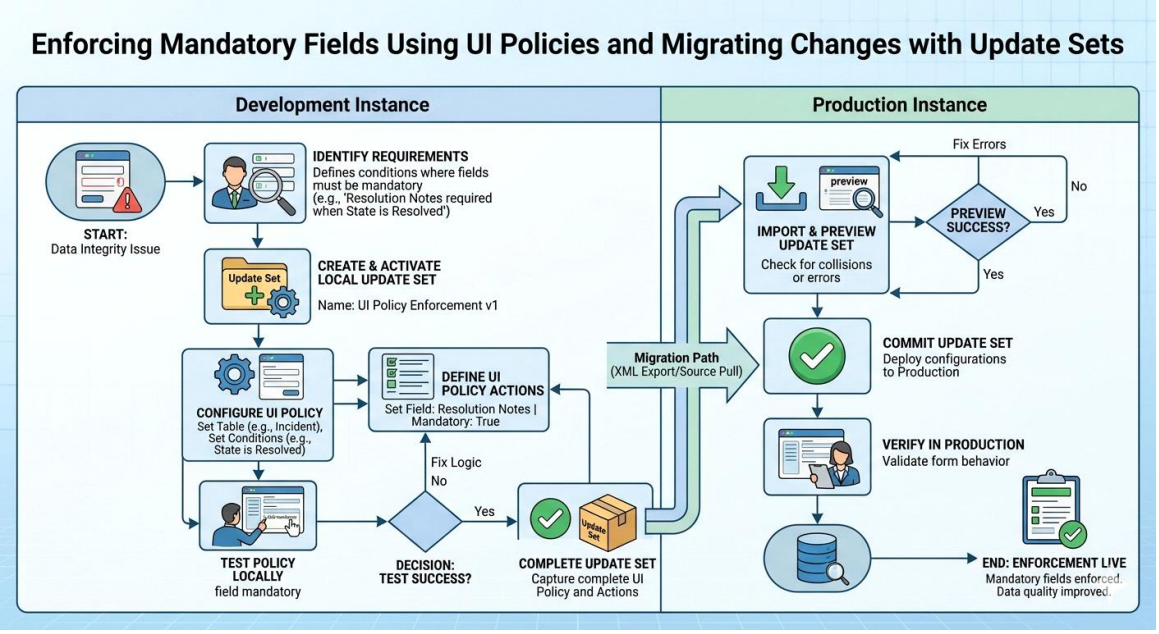
To address this problem, the following technical components must be executed:

1. Logic Configuration (UI Policies)

- **Condition Building:** Define the specific criteria (e.g., `State | is | Resolved`) that trigger the policy.
- **UI Policy Actions:** Set field attributes—specifically **Mandatory**, **Visible**, or **Read Only**—to `True` or `False`.
- **Client-Side Execution:** Ensure the logic runs in the browser to provide immediate feedback to the user without needing a server-side save.

2. Lifecycle Management (Update Sets)

- **Capture:** Initialize a "Local Update Set" to record every change made to the UI Policies and Actions.
- **Validation:** Review the Update Set for errors or "Customer Updates" to ensure no dependencies are missed.
- **Migration:** Close the set, export it (or pull it via Update Sources), and commit it to the target environment.

Expected Outcomes

- **Data Accuracy:** 100% compliance for critical fields during specific process stages.
- **Improved UX:** Forms remain clean and only show mandatory requirements when they are contextually relevant.
- **Auditability:** A clear record of who changed the form logic and when it was deployed via the Update Set history.